# CSC 648 / CSC 848 — Section 03

## Milestone 1 — Version 2

## Home4U

**AI-Assisted, Budget-Aware Interior Style Matching Platform**

Team Number: 4

Team Members & Roles:
Caleb Ponce — Team Lead / System Architecture
Tyler Morris — Backend & AI Integration
Christopher Quach — Frontend Development
Mason Lee — Data Modeling & Scoring Engine
Dias Almat — Technical Writer

Team Lead Email: cponce8@sfsu.edu
Date: February 19, 2026
Version: 2.0

---

# REVISION HISTORY

| Version | Date | Author(s) | Description |
|---|---|---|---|
| 0.1 | Feb 1, 2026 | Team | Initial draft |
| 1.0 | Feb 8, 2026 | Team | Version 1 submission |
| 2.0 | Feb 19, 2026 | Team | Revised based on instructor feedback; completed use cases, diagrams, functional requirements, competitive analysis |

---

# 1. Executive Summary

Home4U is a web-based interior style guidance platform designed to bridge the gap between visual inspiration and actionable transformation. Existing platforms such as Pinterest, Houzz, and Wayfair provide visual inspiration or shopping access, but they do not translate aesthetic styles into structured, budget-aware recommendations tailored to a user's actual space.

Home4U enables renters and first-time apartment dwellers to upload a room image, select desired interior styles (e.g., Modern, Japandi, Industrial), and receive AI-assisted tag suggestions describing current room characteristics. Using a weighted resemblance scoring algorithm, the system computes similarity between the user's room and the selected style(s). Based on this score, Home4U generates prioritized recommendations that maximize aesthetic impact within a defined budget.

Unlike competitors, Home4U integrates explainable scoring, human-validated AI tagging, multi-style comparison, and budget-aware prioritization within a single structured workflow.

The system architecture consists of a React frontend, FastAPI backend (Python 3.12), PostgreSQL relational database, and OpenAI Vision API for tag suggestion, deployed on AWS EC2 behind an Nginx reverse proxy secured with SSL/TLS.

---

# 2. Main Use Cases

## 2.1 Actors

- **User (Renter)**
- **Administrator**

---

## 2.2 Structured Use Cases

---

### UC-01: Create Account

**Primary Actor:** User
**Preconditions:** User is not authenticated
**Postconditions:** User account is created and stored

**Main Flow:**

1. User selects “Sign Up”.
2. User enters email and password.
3. System validates input format.
4. System hashes password.
5. System stores user record.
6. System confirms account creation.

**Alternative Flow:**

- 3a: Email already exists → system displays error.

---

### UC-02: Create Room Project

**Primary Actor:** User
**Preconditions:** User authenticated
**Postconditions:** RoomProject created

**Main Flow:**

1. User selects “Create Room”.
2. User enters room type.

3. System creates RoomProject record.
4. System confirms creation.

---

## UC-03: Upload Room Image

**Primary Actor:** User
**Preconditions:** RoomProject exists
**Postconditions:** Image stored and linked

**Main Flow:**

1. User uploads image.
2. System validates file type.
3. System stores image reference.
4. System associates image with RoomProject.

---

## UC-04: Select Target Style

**Primary Actor:** User
**Preconditions:** RoomProject exists
**Postconditions:** Style(s) selected

**Main Flow:**

1. User selects one or more styles.
2. System associates styles with RoomProject.

---

## UC-05: Confirm Tags

**Primary Actor:** User
**Preconditions:** Image uploaded
**Postconditions:** Confirmed tags stored

**Main Flow:**

1. System generates suggested tags.
2. User reviews suggestions.
3. User edits or confirms tags.
4. System stores confirmed RoomTags.

---

## UC-06: Compute Resemblance Score

**Primary Actor:** System
**Preconditions:** Confirmed tags exist
**Postconditions:** Score stored

**Main Flow:**

1. System retrieves StyleTag weights.
2. System compares RoomTags with StyleTags.
3. System calculates normalized score.
4. System stores ResemblanceScore.

---

## UC-07: Compare Multiple Styles

**Primary Actor:** User
**Preconditions:** Multiple styles selected
**Postconditions:** Comparison displayed

**Main Flow:**

1. System computes score for each style.
2. System displays comparison view.

---

## UC-08: Input Budget Constraint

**Primary Actor:** User
**Preconditions:** Score calculated
**Postconditions:** Budget stored

**Main Flow:**

1. User inputs budget.
2. System validates amount.
3. System stores budget constraint.

---

## UC-09: Generate Recommendations

**Primary Actor:** System
**Preconditions:** Budget and score available
**Postconditions:** Recommendations displayed

**Main Flow:**

1. System identifies missing style tags.
2. System ranks improvements by impact-to-cost ratio.
3. System displays prioritized recommendations.

---

### UC-10: Manage Style Definitions (Admin)

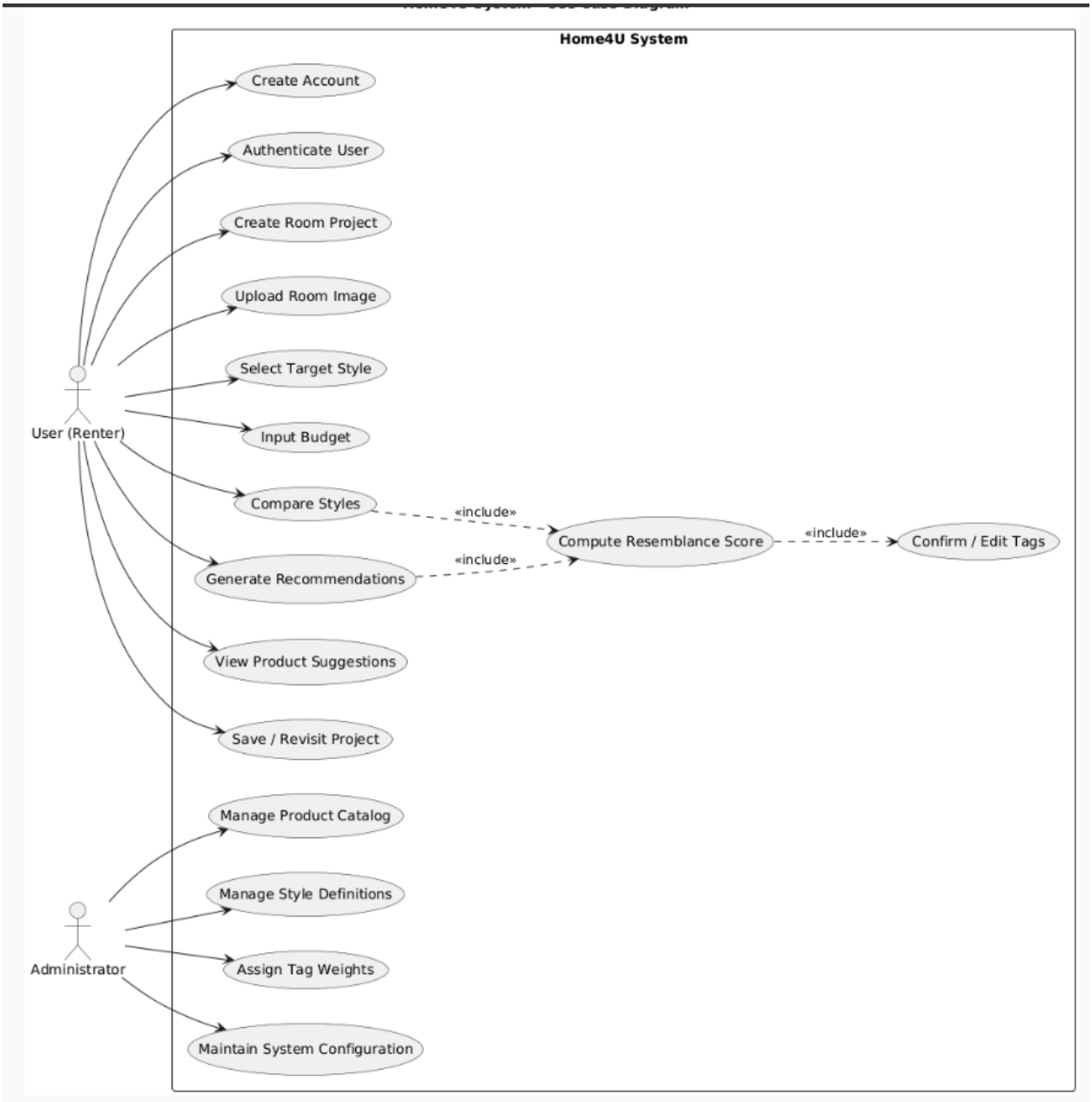
**Primary Actor:** Administrator
**Preconditions:** Admin authenticated
**Postconditions:** Style data updated

**Main Flow:**

1. Admin edits style definition.
2. Admin modifies tag weights.
3. System updates database.

---

## 2.3 Use Case Diagram




Actors:

- User
- Administrator

System boundary:

- Home4U System

Use cases inside system:

- Create Account
- Upload Image
- Select Style
- Confirm Tags
- Compute Score
- Generate Recommendations
- Manage Styles

---

# 3. Main Data Items and Entities

Each entity includes attributes and relationships.

## 3.1 User

- user_id (PK)
- email
- password_hash
- created_at

Relationship:

- One-to-many with RoomProject

## 3.2 RoomProject

- room_id (PK)
- user_id (FK)
- room_type
- budget

Relationship:

- One-to-many with RoomTag

## 3.3 Style

- style_id (PK)
- name
- description

## 3.4 Tag

- tag_id (PK)
- name

## 3.5 StyleTag

- style_id (FK)
- tag_id (FK)

- weight

## 3.6 RoomTag

- room_id (FK)
- tag_id (FK)

## 3.7 ResemblanceScore

- score_id (PK)
- room_id (FK)
- style_id (FK)
- score_value

## 3.8 Recommendation

- recommendation_id (PK)
- room_id (FK)
- description
- priority_score

## 3.9 ProductItem

- product_id (PK)
- style_id (FK)
- name
- estimated_cost
- url

---

# 4. Functional Requirements

## 4.1 User Requirements

USR-01: System shall allow user registration.
USR-02: System shall authenticate users securely.
USR-03: System shall allow creation of RoomProject.
USR-04: System shall allow image upload.
USR-05: System shall allow style selection.
USR-06: System shall allow tag confirmation.
USR-07: A user shall create many room projects.
USR-08: A user shall submit many feedback records.

---

## 4.2 RoomProject Requirements

RP-01: System shall create RoomProject records.
RP-02: System shall associate RoomProject with User.
RP-03: System shall store budget constraint.
RP-04: A room project shall belong to at most one user.
RP-05: A room project shall have at least one room image.
RP-06: A room project shall have at least one room dimension.
RP-07: A room project shall contain at least one room furniture record.
RP-08: A room project shall contain at least one room object record.
RP-09: A room project shall be associated with at least one room tag.
RP-10: A room project shall have at least one resemblance score.
RP-11: A room project shall have at most one budget plan.

---

## 4.3 Style Requirements

ST-01: System shall store predefined styles.
ST-02: Admin shall modify style definitions.
ST-03: A style shall be associated with many tags.
ST-04: A style shall have many resemblance scores.
ST-05: A style shall be associated with many product items.

## 4.4 Scoring Requirements

SC-01: System shall compute weighted resemblance score.
SC-02: System shall normalize scores to 0–100.
SC-03: System shall support multi-style comparison.

---

## 4.5 Recommendation Requirements

REC-01: System shall generate prioritized recommendations.
REC-02: System shall rank recommendations by impact-to-cost ratio.

---

## 4.6 Room Image Requirements

IMG-01: A room image shall belong to one room project.

---

## 4.7 Room Dimension Requirements

DIM-01: A room dimension record shall belong to exactly one room project.

---

## 4.8 Room Furniture Requirements

FUR-01: A room furniture record shall belong to exactly one room project.

---

## 4.9 Room Object Requirements

OBJ-01: A room object record shall belong to exactly one room project.
OBJ-02: A room object record shall be associated with at least one tag.

---

## 4.10 Tag Requirements

TAG-01: A tag shall be associated with many styles.
TAG-02: A tag shall be associated with many room projects.
TAG-03: A tag shall be associated with many room objects.

---

## 4.8 Product Item Requirements

ITM-01: A product item shall belong to exactly one vendor.
ITM-02: A vendor shall supply many product items.
ITM-03: A product item shall belong to exactly one product category.
ITM-04: A product category shall classify many product items.
ITM-05: A product item shall be associated with at most one style.
ITM-06: A product item shall appear in many recommendation items.
ITM-07: A product item shall be allocated in many budget allocations.

---

## 4.8 Vendor Requirements

VEN-01: A vendor shall supply many product items.
VEN-02: A product item shall belong to exactly one vendor.

---

## 4.8 Budget Plan Requirements

BUD-01: A budget plan shall belong to exactly one room project.
BUD-02: A budget plan shall be associated with at least one product item.

---

## 4.8 Feedback Requirements

FED-01:  A feedback record shall belong to exactly one user.

---

# 5. Non-Functional Requirements

## 5.1 Performance

NFR-P-01: Scoring shall complete within 5 seconds.

---

## 5.2 Reliability

NFR-R-01: System shall maintain 99% uptime.

---

## 5.3 Security

NFR-S-01: Passwords shall be hashed and salted.
NFR-S-02: JWT authentication shall be implemented.
NFR-S-03: All communication shall occur over HTTPS using SSL/TLS.

---

## 5.4 Usability

NFR-U-01: Interface shall be responsive.

---

## 5.5 Scalability

NFR-SC-01: System shall support concurrent users.

---

## 5.6 Maintainability

NFR-M-01: Code shall follow PEP8 guidelines.
NFR-M-02: Functions shall include docstrings.

---

## 5.7 Coding Standards

CS-01: Backend shall use Python 3.12.
CS-02: REST endpoints shall follow consistent naming conventions.
CS-03: React 18 functional components required.
CS-04: Git branching strategy must be followed.
CS-05: No hardcoded secrets permitted.

---

# 6. Competitive Analysis

## 6.1 Table 1 – Feature Comparison

| Product | Personalization | Style Scoring | Budget Filter | Shopping Integration |
|---|---|---|---|---|
| Pinterest | No | No | No | Indirect |
| Houzz | Limited | No | No | Yes |
| IKEA Planner | Limited | No | Limited | Yes |
| Havenly | Yes | No | No | Yes |
| Wayfair Ideas | Limited | No | No | Yes |

---

## 6.2 Table 2 – Weakness & Opportunity Matrix

| Competitor | Weakness | Opportunity for Home4U |
|---|---|---|
| Pinterest | No structured recommendations | Provide explainable scoring |
| Houzz | No scoring system | Add measurable similarity index |
| IKEA Planner | Store-specific | Multi-brand neutrality |
| Havenly | Paid service | Self-guided free alternative |
| Wayfair | No personalization engine | AI-assisted personalization |

---

## 6.3 Analytical Summary

Existing platforms focus either on inspiration or commerce but lack explainable personalization. None provide a quantitative resemblance scoring system combined with budget-aware prioritization. Home4U differentiates itself by integrating structured scoring, AI-assisted tagging, and cost-impact ranking into a single user workflow.

---

## 6.4 Target Audience Validation

A preliminary validation exercise was conducted with six renters aged 21–30. Five reported difficulty translating visual inspiration into actionable steps within a fixed budget. Four expressed interest in a scoring system that quantifies style similarity. These findings support the need for structured, budget-aware personalization.

---

# 7. Project Readiness Checklist

☑ Meeting schedule established
☑ Roles assigned
☑ Tools selected
☑ Repository organized
☑ Python version standardized
☑ SSL planned

---

# 8. High-Level Architecture and Technologies

Frontend (React 18)
→ FastAPI (Python 3.12)
→ PostgreSQL 16
→ OpenAI Vision API
→ Nginx Reverse Proxy
→ AWS EC2 (Ubuntu 22.04 LTS)
→ SSL via Let’s Encrypt

---

# 9. Team Contributions

The team divided responsibilities based on technical strengths while ensuring collaborative review of all major deliverables. Each member was responsible for primary ownership of specific sections and technical components.

## Caleb Ponce — Team Lead / System Architecture

**Authored and finalized:**

- Section 1: Executive Summary
- Section 8: High-Level Architecture and Technologies
- Architecture stack definition (React 18 → FastAPI → PostgreSQL → OpenAI Vision API → Nginx → AWS EC2 → SSL)

**Defined and structured:**

- All 10 Structured Use Cases (UC-01 through UC-10)
- System boundary and actor definitions in Section 2.1
- Section 2.3 Use Case Diagram (page 6)

**Created Functional Requirements:**

- SC-01, SC-02, SC-03 (Scoring Requirements)
- REC-01, REC-02 (Recommendation Requirements)
- NFR-S-01, NFR-S-02, NFR-S-03 (Security Requirements)
- CS-01 through CS-05 (Coding Standards)

**Reviewed and approved:**

- Entity structure consistency across Section 3

- Competitive differentiation summary (Section 6.3)

---

# Tyler Morris — Backend & AI Integration

**Defined backend-related use cases:**

- UC-05 (Confirm Tags)
- UC-06 (Compute Resemblance Score)
- UC-09 (Generate Recommendations)
- UC-10 (Manage Style Definitions)

**Authored Functional Requirements:**

- USR-02 (User Authentication)
- SC-01 (Weighted resemblance scoring logic)
- SC-02 (Score normalization to 0–100)
- REC-01 (Recommendation generation logic)
- REC-02 (Impact-to-cost ranking algorithm)

**Specified:**

- OpenAI Vision API tag generation workflow
- FastAPI endpoint behavior for:
    - Image upload
    - Score computation
    - Recommendation generation
- JWT authentication requirement (NFR-S-02)

---

# Christopher Quach — Frontend Development

**Defined user-facing use cases:**

- UC-01 (Create Account)
- UC-02 (Create Room Project)
- UC-03 (Upload Room Image)
- UC-04 (Select Target Style)
- UC-07 (Compare Multiple Styles)
- UC-08 (Input Budget Constraint)

**Authored Functional Requirements:**

- USR-01 (User registration)
- USR-03 (RoomProject creation)
- USR-04 (Image upload)
- USR-05 (Style selection)
- USR-06 (Tag confirmation interface)

**Defined Non-Functional Requirements:**

- NFR-U-01 (Responsive interface)
- CS-03 (React 18 functional component requirement)

**Created:**

- Frontend workflow alignment with use case diagram (page 6)
- Multi-style comparison UI specification

---

# Mason Lee — Data Modeling & Scoring Engine

**Authored Section 3 (Data Entities):**

- 3.1 User
- 3.2 RoomProject
- 3.3 Style
- 3.4 Tag
- 3.5 StyleTag
- 3.6 RoomTag
- 3.7 ResemblanceScore
- 3.8 Recommendation
- 3.9 ProductItem

**Defined relationships and key structures:**

- PK/FK constraints
- Many-to-one and one-to-many mappings
- StyleTag weight attribute design
- ResemblanceScore entity structure

**Created Functional Requirements:**

- RP-01, RP-02, RP-03 (RoomProject Requirements)
- ST-01, ST-02 (Style Requirements)

**Specified:**

- Weighted scoring data schema
- Normalization storage model

---

# Dias Almat — Technical Writer

**Authored and formatted:**

- Section 6: Competitive Analysis
    - Table 1 – Feature Comparison
    - Table 2 – Weakness & Opportunity Matrix
    - Section 6.3 Analytical Summary
    - Section 6.4 Target Audience Validation

**Authored Non-Functional Requirements:**

- NFR-P-01 (Performance – 5 second scoring requirement)
- NFR-R-01 (99% uptime requirement)
- NFR-SC-01 (Scalability requirement)
- NFR-M-01, NFR-M-02 (Maintainability requirements)

**Prepared:**

- Revision History table
- Project Readiness Checklist (Section 7)
- Document formatting, consistency review, and final submission compilation